Real-Time Chat App

Implementation and Learning Outcomes — Built with FastAPI & WebSockets

Project Workflow

1. User enters a username and room name.
2. FastAPI processes the form submission.
3. The user joins the selected chat room.
4. A WebSocket connection is established.
5. Messages are broadcast only to users in the same room.
6. Each message displays: username, timestamp, and a unique username colour.

Challenges Solved

- Virtual environment configuration
- FastAPI installation issues
- Form submission errors
- Template rendering issues
- WebSocket configuration
- Connection management
- Message broadcasting
- JSON message formatting

Skills Gained

- FastAPI application development
- REST routing
- HTML form handling
- Jinja2 template rendering
- WebSocket programming
- Real-time communication
- Git and GitHub version control
- Debugging Python applications

Future Enhancements

- User authentication
- Chat history using SQLite or PostgreSQL
- Online user list
- Emojis and file sharing
- Private messaging
- Improved UI with Bootstrap or Tailwind CSS

Conclusion

This project provided practical experience in building a real-time web application using FastAPI and WebSockets. It strengthened backend development skills, improved debugging abilities, and demonstrated how real-time communication works in modern web applications.

Working through issues such as environment setup, WebSocket connection handling, and message broadcasting gave hands-on insight into how real-time systems are designed and debugged in practice. The result is a functional, room-based chat application with a clear path toward further features such as authentication, persistent history, and a richer user interface.